



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА У
НОВОМ САДУ



Никола Велемир

ПоштаР

ЗАВРШНИ РАД

Основне академске студије

Нови Сад, 2026



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, РБР:	
Идентификациони број, ИБР:	
Тип документације, ТД:	Монографска документација
Тип записа, ТЗ:	Текстуални штампани материјал
Врста рада, ВР:	Дипломски - бечелор рад
Аутор, АУ:	Никола Велемир
Ментор, МН:	Др Никола Лубурић, ванредни професор
Наслов рада, НР:	ПоштаР
Језик публикације, ЈП:	српски/ћирилица
Језик извода, ЈИ:	српски/енглески
Земља публикавања, ЗП:	Република Србија
Уже географско подручје, УГП:	Војводина
Година, ГО:	2026
Издавач, ИЗ:	Ауторски репринт
Место и адреса, МА:	Нови Сад, трг Доситеја Обрадовића 6
Физички опис рада, ФО: (поглавља/страна/ цитата/табела/слика/графика/прилога)	3/21/5/0/1/0/2
Научна област, НО:	Електротехничко и рачунарско инжењерство
Научна дисциплина, НД:	Примењене рачунарске науке и информатика
Предметна одредница/Кључне речи, ПО:	Шаблон, завршни рад, упутство
УДК	
Чува се, ЧУ:	У библиотеци Факултета техничких наука, Нови Сад
Важна напомена, ВН:	
Извод, ИЗ:	Овај документ представља упутство за писање завршних радова на Факултету техничких наука Универзитета у Новом Саду. У исто време је и шаблон за Typst.
Датум прихватања теме, ДП:	
Датум одбране, ДО:	01.01.2025
Чланови комисије, КО:	Председник: Др Петар Петровић, ванредни професор
	Члан: Др Марко Марковић, доцент
	Члан:
Члан, ментор:	Др Никола Лубурић, ванредни професор
	Потпис ментора



UNIVERSITY OF NOVI SAD • FACULTY OF TECHNICAL SCIENCES
21000 NOVI SAD, Trg Dositeja Obradovića 6

KEY WORDS DOCUMENTATION

Accession number, ANO :	
Identification number, INO :	
Document type, DT :	Monographic publication
Type of record, TR :	Textual printed material
Contents code, CC :	
Author, AU :	Nikola Velemir
Mentor, MN :	Nikola Luburić, Phd., assoc. professor
Title, TI :	PoshtaR
Language of text, LT :	Serbian
Language of abstract, LA :	Serbian/English
Country of publication, CP :	Republic of Serbia
Locality of publication, LP :	Vojvodina
Publication year, PY :	2026
Publisher, PB :	Author's reprint
Publication place, PP :	Novi Sad, Dositeja Obradovica sq. 6
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	3/21/5/0/1/0/2
Scientific field, SF :	Electrical and Computer Engineering
Scientific discipline, SD :	Applied computer science and informatics
Subject/Key words, S/KW :	Template, thesis, tutorial
UC	
Holding data, HD :	The Library of Faculty of Technical Sciences, Novi Sad
Note, N :	
Abstract, AB :	This document provides guidelines for writing final theses at the Faculty of Technical Sciences, University of Novi Sad. At the same time, it serves as a Typst template.
Accepted by the Scientific Board on, ASB :	
Defended on, DE :	01.01.2025
Defended Board, DB :	President: Petar Petrović, Phd., assoc. professor
	Member: Marko Marković, Phd., asist. professor
	Member:
Member, Mentor:	Nikola Luburić, Phd., assoc. professor
	Mentor's sign



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

Кандидат

TBD

Садржај

1	Увод	1
1.1	О библиотеци	1
1.2	Типична реализација архитектуре апликација	1
1.2.1	Опис типичне архитектуре	1
1.2.2	Проблеми и ограничавајући фактори	2
1.3	<i>Mediator</i> софтверски образац	2
2	Могућа и постојећа решења	3
2.1	Наивно решење	3
2.2	Постојећа решења у <i>Java</i> екосистему	3
2.2.1	<i>Axon Framework</i>	4
2.2.2	<i>PipelinR</i>	4
2.3	Постојећа решења из других екосистема	5
2.3.1	<i>MediatR</i> у <i>C#</i>	5
2.4	Опште мане имплементација <i>Mediator</i> обрасца у различитим екосистемима	5
2.4.1	Архитектуралне импликације	5
2.4.2	Искусствене импликације	6
3	Закључак	7
	Списак слика	9
	Списак листинга	11
	Списак коришћених скраћеница	13
	Списак коришћених појмова	15
	Биографија	17
	Литература	19
	Грешке у литературним наводима	21

1.1 О библиотеци

PoshtaR је првенствено имплементација *Mediator* софтверског обрасца, намењена за *Java* програмски језик и његове екосистеме. Интеграцију са специфичним јава екосистемима и њиховим радним оквирима, *PoshtaR* постиже путем адаптера намењеног за специфичан радни оквир. Применом *Mediator* софтверског обрасца се постиже раздвајање пословне логике, смањење зависности између различитих компоненти и њихово лабаво повезивање.

1.2 Типична реализација архитектуре апликација

1.2.1 Опис типичне архитектуре

Јава екосистеми често реализују организацију компоненти апликације путем слојевите архитектуре (*Layered architecture*). Овако подељена архитектура најчешће врши поделу компоненти по слојевима:

- **Контролерски слој (*Controller layer*)** - Захтеви примљени путем неког комуникационог протокола се прихватају путем контролерског слоја, који диктира даљу делегацију компонентама које репрезентују пословну логику.
- **Сервисни слој (*Service layer*)** - Сервисни слој репрезентује пословну логику система. Пословна логика система садржи рачунања, трансакције и промене стања система која репрезентују извршавање пословних циљева за које је систем намењен.
- **Складишни слој (*Repository layer*)** - Складишни слој енкапсулира логику промене и приказа података који су релевантни за реализацију пословних циљева система. Подаци су складиштени унутар неког типа базе података.
- **Слој модела (*Model layer*)** - Слој модела репрезентује ентитете (ствари, актере или чињенице) специфичне за домен проблема пословне логике којима се систем бави.

Најчешће варијације унутар овог приступа су:

- **Хоризонтално подељена архитектура (*Horizontal Slice Architecture*)** - Организација кода се примарно дели према претходно наведеним техничким слојевима (пакети *controllers*, *services*, *repositories*), док се унутар сваког слоја креирају подсклопови везани за конкретне функционалности или ентитете пословног домена (*feature*).
- **Вертикално подељена архитектура (*Vertical Slice Architecture*)** - Организација кода се примарно дели по функционалностима система (пакети *users*, *orders*, *payments*), док свака функционалност унутар себе садржи сопствене компоненте свих потребних техничких слојева.

1.2.2 Проблеми и ограничавајући фактори

Иако су слојевита и њене изведене архитектуре широко прихваћене у *Java* екосистему, оба приступа носе изазове у развоју и одржавању софтвера:

- **Проблем чврстог повезивања у хоризонталној архитектури** – У хоризонтално подељеној архитектури, сервисни слој често постаје преобиман (тзв. *God Service* класе). Пошто сервиси директно позивају друге сервисе и складишта ради реализације комплексних пословних токова, ствара се мрежа чврстих међусобних зависности (*tight coupling*). Ово отежава писање изолованих јединичних тестова, а свака измена логике једног сервиса ризикује споредне ефекте у остатку система.
- **Проблем дуплирања и изолације у вертикалној архитектури** – С друге стране, вертикално подељена архитектура тежи да потпуно изолије функционалности, што у практичној реализацији често доводи до дуплирања заједничке пословне логике. Поред тога, када је за извршење пословног циља неопходна координација између више компоненти, јавља се проблем управљања њиховом међусобном комуникацијом без нарушавања саме идеје вертикалне изолације.
- **Сложеност контроле токова података** – У оба модела, како систем расте, праћење логике кроз слојеве и функционалности постаје све сложеније. Измена једног корака у пословном процесу неретко захтева модификације кода на више места. Тиме се повећава могућност људске грешке при одржавању.

Потреба за смањењем директних зависности, централизацијом контроле комплексних токова и очувањем чисте организације кода, представљају главни мотив за увођење софтверских образаца понашања попут **Mediator** (Посредник) обрасца.

1.3 Mediator софтверски образац

Образак **Mediator** (посредник) решава наведене недостатке увођењем централне компоненте која преузима улогу посредника у комуникацији између свих осталих компонената система. Уместо да сервисни слојеви или вертикалне кришке директно позивају једни друге и тако стварају мрежу чврстих зависности, све компоненте комуницирају искључиво преко посредничке компоненте. На овај начин се постиже:

- **Лабаво повезивање (*Loose Coupling*)** – Компоненте постају потпуно одвојене једна од друге, јер знају само за интерфејс Медијатора. Овиме се елиминише проблем спреге сервиса и омогућава да се појединачне компоненте развијају, тестирају и мењају потпуно независно.
- **Ефикасна координација вертикалних кришки** – Медијатор постаје магистрала за поруке или догађаје (*Event/Command Bus*). Када једна функционалност мора да иницира акцију у другој, она једноставно објављује команду или догађај. Медијатор води рачуна о томе ко треба да обради поруку, чувајући притом чисту изолацију између слојева.
- **Централизација и лакше одржавање пословних токова** – Комплексна логика оркестрације и међусобне интеракције компонената премешта се из сервиса у посредника. Овиме се олакшава праћење тока података кроз систем, смањује дуплирање кода и поједностављује увођење нових функционалности без ризика од неочекиваних споредних ефеката на постојеће слојеве.

Могућа и постојећа решења

Опште решење реализације Медијатор обрасца је приказано сликом 1. Општа реализација обрасца подразумева увођење абстракције (интерфјес *Mediator*) које је асоцирана са конкретном имплементацијом логики Медијатор објекта (*ConcreteMediator*). Приликом позива методе за примање одређеног захтева (нпр. *dispatch*), конкретна имплементација обрасца је у стању да констатује којој компоненти је неопходно делегирати даље извршавање (нпр *ConcreteColleague1*).



- **Пораст величине посредничког објекта (*God Object Problem*)** – Како систем расте и креирају се нове функционалности, јавља се потреба за проширењем кода посредничког објекта. Ово у најужем смислу обухвата креирање нових *Colleague* објеката и њихову регистрацију. У радним оквирима *Java* екосистема (попут *Spring* радног оквира), ово се манифестује кроз стално инјектовање нових зависности унутар посредничког објекта.

- ## 2.2 Постојећа решења у *Java* екосистему

3

2.2.1 Axon Framework

Axon Framework је софтверски оквир намењен изградњи дистрибуираних Java система. оквир се у великој мери ослања на архитектуру вођену догађајима (*Event-Driven Architecture*) и принцип *Event Sourcing*-а.

У контексту *Mediator* обрасца, *Axon* елиминише проблем монолитног и преоптерећеног посредничког објекта тако што уводи децентрализоване магистрале порука (*Message Buses*). Уместо једне конкретне класе која зна за све компоненте, систем се заснива на три кључна типа порука, за које постоје специјализовани посредници:

- **Command Bus** – Посредник задужен за рутирање команди (намера за променом стања система) до њихових специфичних обрађивача (`@CommandHandler`). Свака команда има тачно једног одредишног примаоца, чиме се постиже чиста сегрегација пословне логике.
- **Event Bus** – Након што се стање система измени, генерише се догађај који се путем ове магистрале дистрибуира до свих заинтересованих компоненти (`@EventHandler`). Ово омогућава лабаво повезивање, јер компонента која је иницирала промену нема никакво сазнање о томе које све компоненте реагују на ту промену.
- **Query Bus** – Магистрала намењена рутирању упита за приказ података до одговарајућих обрађивача (`@QueryHandler`), подржавајући и концепте попут тачка-до-тачке, раштрканих (*scatter-gather*) или претплатничких упита.

Главна предност коју *Axon* доноси у односу на наивну имплементацију јесте динамичка регистрација компоненти, односно елиминација потребе за ручном регистрацијом и инјекцијом компоненти. Нови обрађивачи команди или догађаја се региструју путем анотација, без икакве потребе за модификацијом самог посредничког објекта. Овиме се поштује *Open/Closed* принцип. Међутим, највећа мана овог оквира је висока комплексност, велика потреба за савладавањем оквира и инфраструктурно оптерећење. Мане *Axon* оквира условљавају да је оквир често превише комплексно решење за стандардне пословне апликације.

2.2.2 PipelinR

PipelinR је Java библиотека примарно намењена за *Spring* радни оквир. Библиотека за циљ има да омогући чисту имплементацију команди и упита, без комплексне инфраструктуре какву захтевају већи радни оквири попут *Axon*-а. Дизајнирана је по узору на популарне библиотеке из других екосистема (као што је *MediatR* у *.NET* екосистему).

Уместо централног објекта који садржи референце ка свим сервисима, *PipelinR* уводи концепт цевовода (*pipeline*) и заснива се на три основна градивна блока:

- **Command (или Query)** – Представља једноставан објекат који описује намеру или захтев.
- **Handler** – Компонента задужена за извршавање конкретне пословне логике везане за одређену команду.
- **Pipeline** – Сам посреднички објекат (*Mediator*) који прихвата команду и аутоматски проналази и покреће одговарајући *Handler*.

Библиотека не поседује уграђен механизам за аутоматско скенирање класа. Упркос томе, омогућава да се кроз *Spring* конфигурационе класе прикупе сви обрађивачи (`Command.Handler`) из апликативног контекста и проследи главном *Pipeline* објекту.

Након што се ова почетна конфигурација постави, увођење нове функционалности захтева само креирање нових *Command* и *Handler* класа (означених са анотацијом `@Component`). *Spring* ће их аутоматски скенирати и доставити цевоводу, чиме се у потпуности елиминише потреба за ручном инјекцијом нових зависности у сам посреднички објекат и задовољава *Open/Closed* принцип.

Главна мана овог решења је што је библиотека ограничена искључиво на синхрону комуникацију по моделу “један на један” (једна команда – један обрађивач), па није погодна за системе који захтевају асинхронно раштркавање порука или архитектуру вођену догађајима са више претплатника.

2.3 Постојећа решења из других екосистема

2.3.1 *MediatR* у C#

MediatR је једна од најпопуларнијих и најшире прихваћених библиотека у .NET екосистему. Библиотека је постала стандард за имплементацију *Mediator* обрасца и *CQRS* архитектуре у овом окружењу. Својим именом и утицајем библиотека је поставила образац са деривате *Mediator* имплементације у другим програмским језицима.

MediatR се заснива на истом концепту раздвајања порука на команде/упите и њихове обрађиваче. Главни посреднички објекат прихвата поруку и динамички је рутира до одговарајућег обрађивача. Као софтверско решење, он нуди неколико кључних предности у решавању проблема наивне имплементације:

- **Аутоматско скенирање** – За разлику од својих деривата у другим језицима, *MediatR* долази са уграђеним екстензијама за .NET *Dependency Injection* контејнер. То омогућава да се унутар конфигурације апликације позове само једна метода која аутоматски скенира склопове (*assemblies*), проналази све дефинисане обрађиваче и региструје их. Тиме се у потпуности елиминише потреба за модификацијом посредничког објекта и успешно имплементира *Open/Closed* принцип.
- **Разноврсност комуникационих модела** – Поред синхроног “један на један” рутирања, *MediatR* подржава и модел објављивања догађаја вишеструким претплатницима (такозване нотификације), што му омогућава да функционише и као локална магистрала догађаја унутар једне апликације.
- **Цевоводи (*Pipeline Behaviors*)** – Ова функција омогућава креирање пресретача који омогућавају једноставно убацивање попречних функционалности (*cross-cutting concerns*) попут валидације, логовања, кеширања и управљања трансакцијама дуж саме путање захтева, унапређујући тако сегрегацију пословне логике.

2.4 Опште мане имплементација *Mediator* обрасца у различитим екосистемима

2.4.1 Архитектуралне импликације

Иако наведене имплементације решавају недостатке наивног приступа, оне често отварају нове изазове и импликације архитектуралне природе. Неки од проблема везаних за саму архитектуру система су:

- **Ризик од неисправне резолуције и преклапања обрађивача** – Уколико се грешком региструје више обрађивача за исти тип команде или упита, нарушава се стриктна “један-на-један” веза између поруке и њеног јединственог извршиоца. Оваква лоше постављена конфигурација се не уочава током компајлирања, већ се испољава искључиво у време извршавања апликације (*runtime*). То резултује грешкама у пословној логици или бацањем изузетака. Додатно, неки оквири једноставно пребришу претходне обрађиваче и покрену само последњи регистровани.
- **Нарушавање принципа јединствене одговорности (*Single Responsibility Principle*)** – Класа која има улогу обрађивача порука требало би да буде задужена за извршавање логике једног и само једног типа поруке. Међутим, у већини актуелних библиотека не постоји ограничење високог нивоа које би експлицитно забранило да једна класа бива задужена за обраду различитих порука. Дозвољавањем овакве праксе, класа брзо акумулира вишеструке одговорности, чиме се директно крши овај фундаментални принцип објектно-оријентисаног дизајна.
- **Проблем наслеђивања између класа порука** – Уколико један тип поруке (класа) наслеђује други, семантика и логика извршавања постају нејасне. Са становишта архитектуре, поставља се комплексно питање резолуције: да ли поруку изведеног (подређеног) типа треба обрадити искључиво путем обра-

ђивача који је за њу експлицитно дефинисан, или је треба проследити и обрађивачу надтипа, или пак активирати оба у одређеном редоследу. Библиотеке често немају једнообразан одговор на овај проблем, што уноси непредвидивост у рад система.

- **Заобилажење посредничког објекта директном инјекцијом компоненти** – Дизајн заснован на Медијатору захтева да се међусобна интеракција између зависних компоненти спроводи искључиво преко њега. Међутим, у пракси је и даље технички дозвољено да се инстанцирају или директно инјектују друге компоненте унутар једног обрађивача. Експлицитно повезивање компоненти чијом би комуникацијом примарно требало да руководи посредник нарушава основни смисао његовог постојања и архитектуре лабавог повезивања.
- **Непостојање обрађивача за конкретни тип поруке** – Већина имплементација омогућава да се порука пошаље кроз систем чак и када не постоји регистрован обрађивач за њу. Понашање система варира од оквира до оквира: поједине библиотеке ће бацити изузетак и прекинути извршавања, док ће друге једноставно “прогутати” поруку без икаквог обавештења. Креирање објеката порука који немају коњунктивни пар у виду обрађивача је бесмислено.

2.4.2 Искуствене импликације

Многе имплементације Медијатор обрасца су пројектоване тако да буду стриктно везане за одређени софтверски екосистем или радни оквир. Са аспекта инжењера који развија апликацију, овакав дизајн ствара когнитивно оптерећење приликом транзиције на друге технологије. Уместо да се ослони на чисте концепте дизајна, инжењер је приморан да изнова учи инфраструктурне специфичности нове библиотеке. Ово нарушава сигурност у развоју и флексибилност инжењера, јер имплементације често не подражавају универзалне архитектонске принципе, већ прате искључиво парадигму једног конкретног радног оквира.

Глава 3

Закључак

У закључку дајте кратак преглед онога шта урађено, са освртом на проблеме који су решени, предности и мане решења и правце даљег развоја.

Списак слика

Слика 1	Дијаграм опште имплементације <i>Mediator</i> обрасца.	3
---------	---	---

Списак листинга

Списак коришћених скраћеница

Скраћеница	Опис
API	Application Programming Interface (апликациони програмски интерфејс)
AWS	Amazon Web Services (Амазон веб сервиси)
CI/CD	Continuous Integration / Continuous Delivery (континуирана интеграција / континуирана испорука)
CORS	Cross-Origin Resource Sharing (размена ресурса између извора и дестинације различитог порекла)
CSS	Cascading Style Sheets (језик за описивање стилова)
DOM	Document Object Model (објектни модел документа)
DTO	Data Transfer Object (објекат за пренос података)
HTTP	HyperText Transfer Protocol (протокол за пренос хипертекста)
JSON	JavaScript Object Notation (формат за размену података)
JWT	JSON Web Token (сигурносни токен заснован на JSON формату)
RLS	Row-Level Security (сигурност на нивоу реда)
REST	Representational State Transfer (скуп правила за комуникацију између клијента и сервера)
RPC	Remote Procedure Call (позив удаљене процедуре)
SQL	Structured Query Language (структурирани упитни језик)
TLS	Transport Layer Security (безбедност транспортног слоја)
UML	Unified Modeling Language (језик за моделовање дијаграма)
URL	Uniform Resource Locator (јединствени идентификатор и локатор ресурса)
UI	User Interface (кориснички интерфејс)
UUID	Universally Unique Identifier (универзално јединствени идентификатор)
WAL	Write-Ahead Logging (записивање операција унапред)

Списак коришћених појмова

Појам	Објашњење
Асинхрони рад	Рад који се изводи независно од главног тока извршавања, омогућавајући наставак других операција без чекања на његов завршетак.
Bucket (S3)	Логичка јединица за складиштење у AWS S3 сервису, која организује фајлове у облаку.
Read-Only	Режим рада у коме су подаци само за читање, без могућности измене.
Cross-platform	Способност софтвера да се извршава на више различитих оперативних система из истог кода.
Connection pool	Механизам за управљање и поновну употребу веза са базом података како би се побољшале перформансе апликације.

Биографија

Овде написати своју кратку биографију.

Литература

Грешке у литературним наводима

Овде се налази списак грешака у литературним наводима које морате исправити у `literatura.bib` фајлу.

1. Референци `pyflies` недостаје поље `urldate`